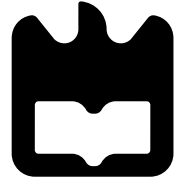


universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Distributed Systems Communication

Eurico Pedrosa <efp@ua.pt>

2nd semester - 2025'26

Communication

Foundations

Communication is the **fundamental mechanism** that allows processes on different machines to exchange information in a distributed system.

To manage the complexity of this interaction, communication is structured through:

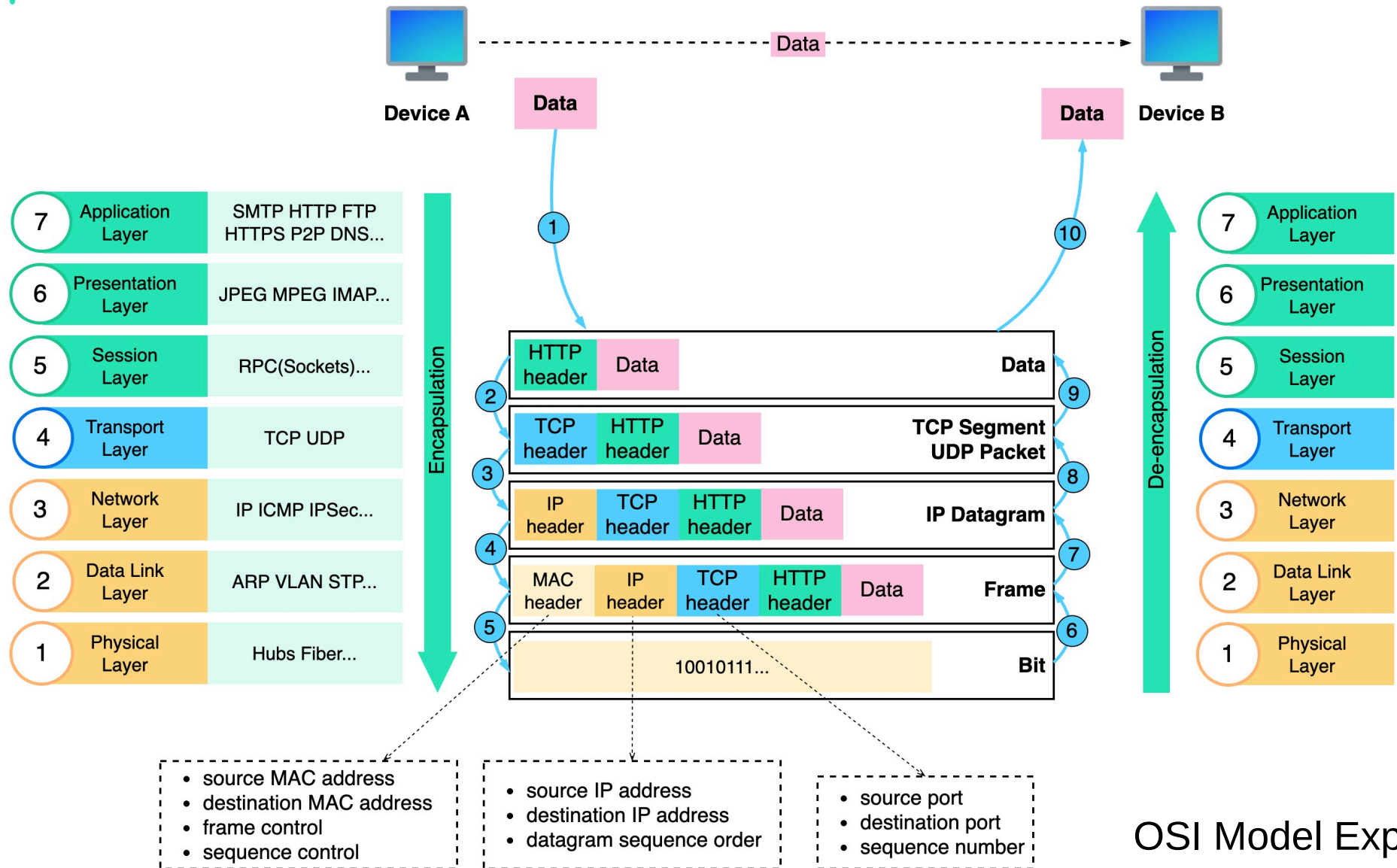
- **Protocols:** Sets of rules governing the format, contents, and meaning of messages.
- **Layering:** Dividing communication tasks into independent levels to ensure modularity.
- **Classification:** Categorizing communication based on persistence and synchronicity.

Foundations: Layered Protocols

Distributed systems rely on layered protocol stacks where each layer provides a specific service to the layer above it. The International Standards Organization (ISO) defined the **Open Systems Interconnection (OSI)** reference model to standardize these layers.

Layer	Primary Responsibility
Application	E-mail, Web access, and file-transfer protocols
Presentation	Independent data representation and syntax
Session	Session management between applications
Transport	Reliable end-to-end communication and real-time streaming
Network	Routing messages and handling congestion (e.g., IP)
Data Link	Grouping bits into frames and error detection
Physical	Standardizing bit representation (0s and 1s) and hardware interfaces

What is OSI model



OSI Model Explained

Foundations: Persistent vs. Transient Communication

A key distinction in communication models is how long the system buffers information.

- **Persistent Communication:**

- The communication system stores a message until it is delivered to the receiver.
- Neither the sender nor the receiver needs to be active at the same time for the transmission to succeed.
- Example: Traditional e-mail systems.

- **Transient Communication:**

- The communication system does not offer long-term storage.
- A message is discarded if the next hop or the receiver is not available.
- Both parties must be up and running for the communication to occur.

Foundations: Asynchronous vs. Synchronous Communication

The synchronization behavior between a sender and a receiver determines how the application's control flow proceeds during communication.

- **Asynchronous Communication:**

- The sender continues its execution immediately after submitting the message
- The message is buffered by the communication system for later delivery

- **Synchronous Communication:**

- The sender blocks until a specific event occurs
- Common synchronization points include:
 - The message has been accepted by the communication system
 - The message has been delivered to the receiver
 - The receiver has processed the message and returned a response (Request-Reply)

Foundations: Connection-Oriented vs. Connectionless

Layers provide two primary types of communication services to applications:

- **Connection-oriented Service** 
 - Modeled after the telephone system; requires establishing a session before data exchange.
 - Parameters are often negotiated during the setup phase.
- **Connectionless Service** 
 - Modeled after the postal system; messages (or packets) are sent independently without prior setup.
 - Each message must contain the full destination address.

Remote Procedure Call

Remote Procedure Call (RPC)

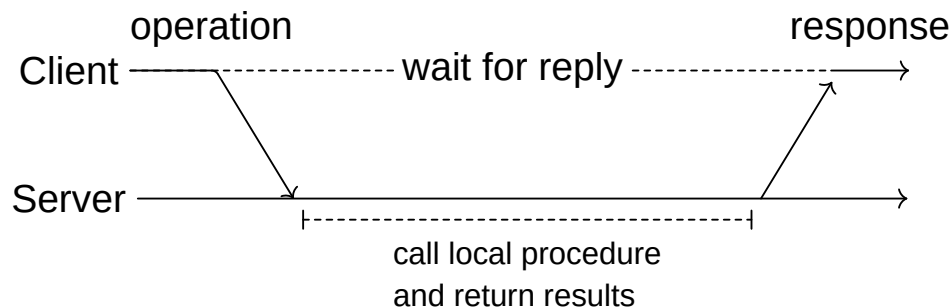
The **Remote Procedure Call (RPC)** is a communication model designed to provide **access transparency** in distributed systems. Its primary objective is to make a procedure call to a remote machine appear identical to a local call within the same process.

- **Concept:** A process on machine A calls a procedure on machine B; the calling process on A is suspended while the procedure executes on B.
- **Transparency:** All message passing is hidden from the programmer, who interacts only with standard procedure signatures.
- **Challenge:** Differences in address spaces, machine architectures, and potential network failures complicate the “local call” illusion.

Remote Procedure Call (RPC): Basic Operation

RPC relies on **stubs** to bridge the gap between address spaces.

- **Client Stub:** A library routine that represents the remote procedure to the client. It packs parameters into a message (marshalling) and waits for a reply.
- **Server Stub:** The server-side counterpart that unpacks the message (unmarshalling) and performs the actual local call to the server procedure.

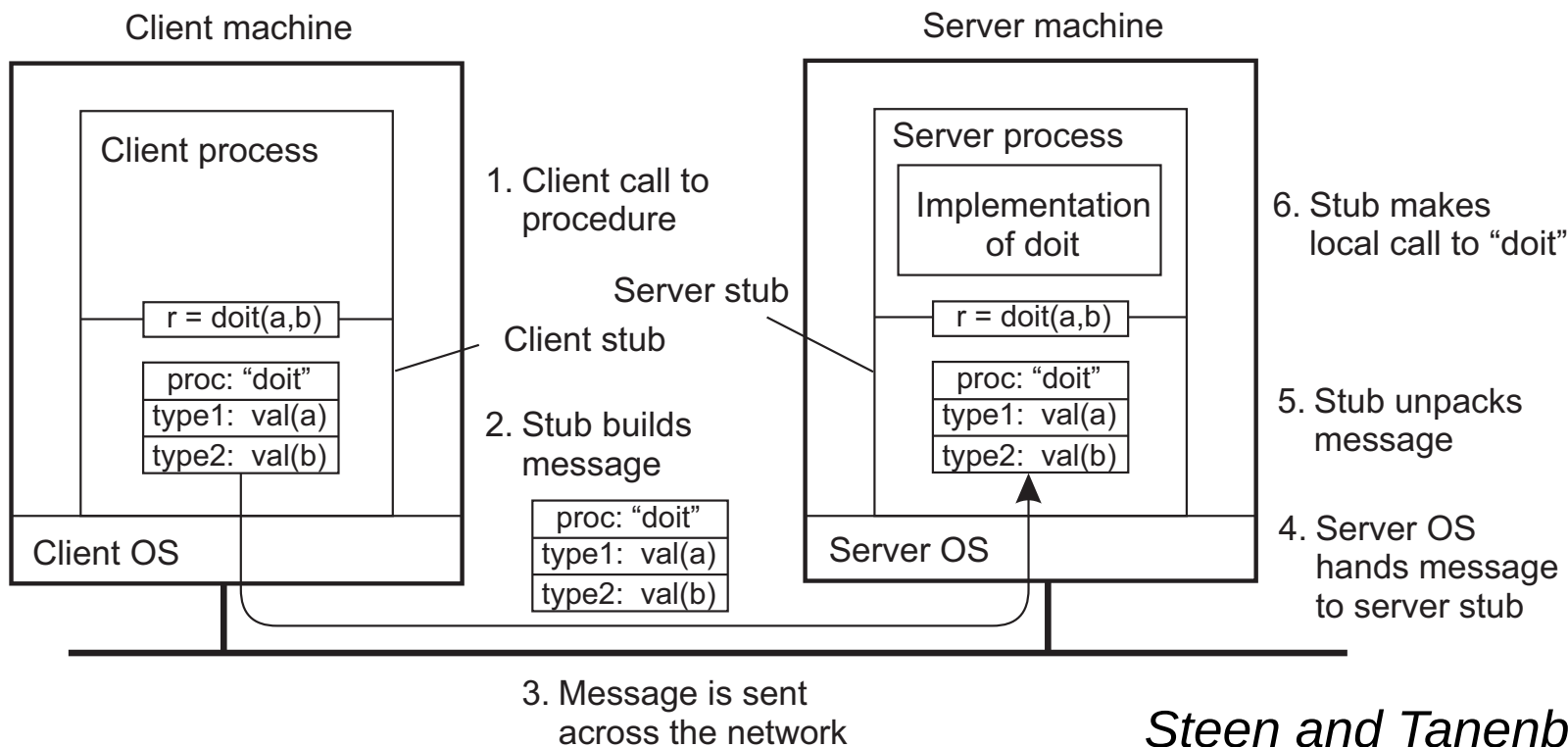


Remote Procedure Call (RPC): Basic Operation

A remote procedure call occurs in the following steps 10:

1. Client procedure calls the client stub
2. Client stub builds a message and calls the local OS
3. Client OS sends the message to the remote OS
4. Remote OS delivers the message to the server stub
5. Server stub unpacks parameters and calls the server procedure
6. Server completes work and returns result to the server stub
7. Server stub packs result in a message and calls local OS
8. Server OS sends message to the client OS
9. Client OS delivers message to the client stub
10. Client stub unpacks result and returns it to the client procedure

Remote Procedure Call (RPC): Basic Operation



Steen and Tanenbaum (2023)

Remote Procedure Call (RPC): Parameter Passing

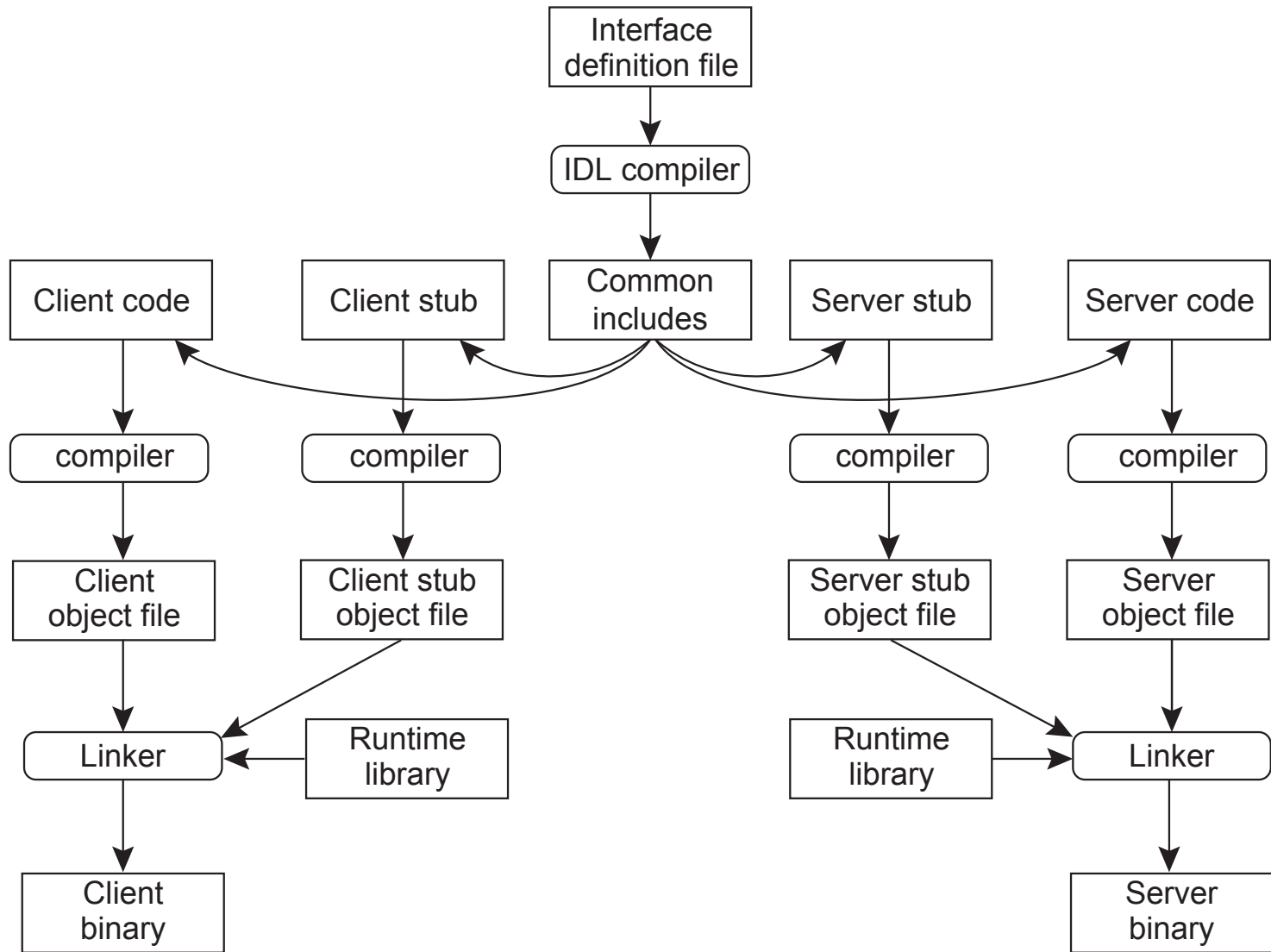
Passing information between machines requires **marshalling**: converting data into a machine-independent format.

- **Data Representation**: Different architectures use different byte orderings Marshalling ensures the server correctly interprets the incoming byte stream.
 - e.g., **Intel's little-endian** vs. **network-standard big-endian**.
- **Pointers & References**: Memory addresses are meaningless in a remote address space.
 - **Solution 1**: Use **copy-by-value/restore**, where the entire data structure is copied to the server and the updated version is copied back to the client.
 - **Solution 2**: Use **global references** that both the client and server understand
 - e.g., file handles

Remote Procedure Call (RPC): Application Support

To automate the creation of stubs and ensure protocol consistency, middleware provides development tools.

- **Interface Definition Language (IDL):** A formal language used to specify the procedure names, parameters, and return types of a service.
- **Stub Generation:** An IDL compiler takes the interface definition and generates:
 - **Client Stubs:** For the client application to link against
 - **Server Stubs (Skeletons):** To interface with the server-side implementation
 - **Runtime Library:** Handles the actual transport, error control, and synchronization of the RPC



Remote Procedure Call (RPC): Variations

Standard RPC is synchronous, but variations improve performance and flexibility.

- **Asynchronous RPC:** The server immediately sends an acknowledgment (ACK) upon receiving the request. The client resumes execution while the server processes the call
- **Deferred Synchronous RPC (Callbacks):** A combination where the client initiates an asynchronous call and later receives the result via a **callback** (a user-defined function invoked when the server finishes)
- **One-way RPC:** The client sends a request and continues immediately without waiting for any acknowledgment from the server. This is risky unless the underlying transport is highly reliable

Remote Procedure Call (RPC): gRPC

gRPC (Google Remote Procedure Call) is a modern, high-performance, open-source RPC framework that can run in any environment. It evolved from the need to connect services in a scalable and efficient manner, particularly in microservices architectures.

- **Transport:** It uses **HTTP/2** for transport, which supports features like bidirectional streaming, flow control, and header compression.
- **Interface Definition:** It relies on **Protocol Buffers** (protobuf) as both its Interface Definition Language (IDL) and its message interchange format.
- **Language Agnostic:** Supports many languages (Java, Python, C++, etc.), allowing a Java client to call a C++ server transparently.

Remote Procedure Call (RPC): gRPC IDL

Protocol Buffers are used to define the structure of the data and the service interface.

- **Strong Typing:** All fields are explicitly typed (int32, string, bool, etc.).
- **Efficiency:** Messages are serialized into a compact binary format, which is much smaller and faster to process than text-based formats like JSON or XML.
- **Backward Compatibility:** Fields can be added or removed without breaking existing services, provided tag numbers are maintained.

Remote Procedure Call (RPC): gRPC IDL Example

A `.proto` file contains the service definition and the request/response message types.

```
syntax = "proto3";  
package distributed_systems;  
// The service definition.  
service Greeter {  
    // Sends a greeting  
    rpc SayHello (HelloRequest) returns (HelloReply) {}  
}  
// The request message containing the user's name.  
message HelloRequest {  
    string name = 1;  
}  
// The response message containing the greetings  
message HelloReply {  
    string message = 1;  
}
```

Remote Procedure Call (RPC): RPC Types gRPC

gRPC supports four types of service methods, providing flexibility for different app needs:

- **Unary RPCs:** The client sends a single request and gets a single response (similar to a normal function call)
- **Server Streaming RPCs:** The client sends one request and the server returns a stream of messages
- **Client Streaming RPCs:** The client sends a stream of messages and the server returns a single response
- **Bidirectional Streaming RPCs:** Both sides send a sequence of messages using a read-write stream

Remote Procedure Call (RPC): gRPC workflow

1. **Define:** Define the service and messages in a `.proto` file
2. **Generate:** Use the protoc compiler to generate client and server stubs in your chosen language (e.g., Java)
3. **Implement:** Implement the server logic by overriding the generated base classes
4. **Call:** The client uses the generated stub to call the server methods as if they were local

Message-Oriented Communication

Message-Oriented Communication

Message-oriented communication serves as an alternative to the Request-Reply pattern of RPC, particularly when the synchronous nature of RPC is inappropriate.

- **Concept:** Communication is based on the exchange of messages rather than RPC
- **Decoupling:** Messaging often allows for looser coupling between sender and receiver, supporting cases where the receiver may not be active when the request is sent
- **Categories:**
 - **Transient:** Messages are stored only as long as the sending and receiving applications are executing.
 - **Persistent:** Messages are stored by the middleware (queues) until they can be delivered, regardless of application state.

Simple Transient Messaging with Sockets

Sockets provide a low-level, message-oriented model offered by the transport layer.

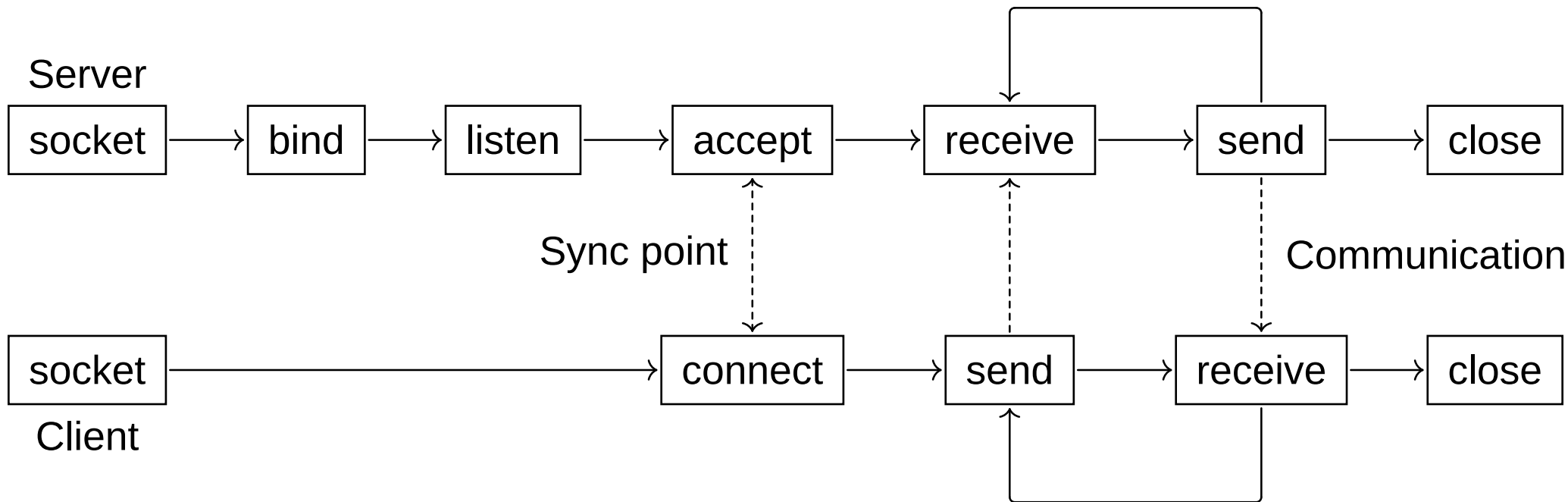
- **Definition:** A socket is a communication endpoint (abstraction over a port) to which an application can write and read data.
- **Mechanism:** Standardized interfaces (like POSIX/Berkeley sockets) allow for portability across different machines.

Simple Transient Messaging with Sockets

Operation	Description
socket	Creates a new communication endpoint.
bind	Attaches a local address/port to the socket.
listen	Defines the maximum number of pending connection requests.
accept	Blocks the server until a connection request arrives.
connect	Client actively attempts to establish a connection.
send/receive	Exchanges data over the established connection.

Simple Transient Messaging with Sockets

Connection-oriented communication pattern using sockets:



Advanced Transient Messaging with Sockets

Advanced messaging layers build upon sockets to simplify network programming and support complex patterns.

- **ZeroMQ Patterns:** ZeroMQ enhances sockets by abstracting connection management and offering specific messaging behaviors:
 - **Request-Reply:** Traditional client-server interaction
 - **Publish-Subscribe:** Multicasting messages from a publisher to multiple subscribers
 - **Pipeline (Push-Pull):** Distributing tasks among a set of workers
- **Message-Passing Interface (MPI):** Designed for high-performance parallel applications.
 - **Group-based:** Assumes communication within a known group of processes.
 - **Advanced Buffering:** Supports various forms of synchronization

Message-Oriented Persistent Communication

Message-oriented persistent communication, primarily realized through **message-queuing systems**, or just **Message-Oriented Middleware (MOM)**, provides intermediate-term storage for messages.

- **Persistence:** Messages are stored by the communication system as long as it takes to deliver them to the receiver.
- **Asynchronous Nature:** Senders can submit a message and continue execution immediately, without waiting for the receiver to be active.
- **Reliability:** Messages are stored in queues and eventually delivered even if the destination was down at the time of sending.

MOM: Message-Queuing Model

In this model, applications interact by inserting messages into specific, named queues rather than calling procedures directly.

- **Private Queues:** Typically, each application has its own private queue which other applications can write to
- **Time Decoupling:** The fundamental benefit is the ability for communication to be decoupled in time – neither party needs to be executing at the same moment
- **Queue Storage:** Messages remain in the queue until they are explicitly removed by a recipient or expire.

MOM: Execution Combinations (Decoupling)

The persistence of the system allows for four distinct execution states between a sender and a receiver:

Sender	Receiver	System Behavior
Active	Active	Immediate transmission and delivery
Active	<i>Passive</i>	System buffers message until receiver becomes active
<i>Passive</i>	Active	Receiver reads messages previously left by the sender
<i>Passive</i>	<i>Passive</i>	System holds message; transmission/delivery occurs later

MOM: Queue Operation (Programming Model)

Message-queuing systems offer a simple, high-level interface to applications for managing message flow.

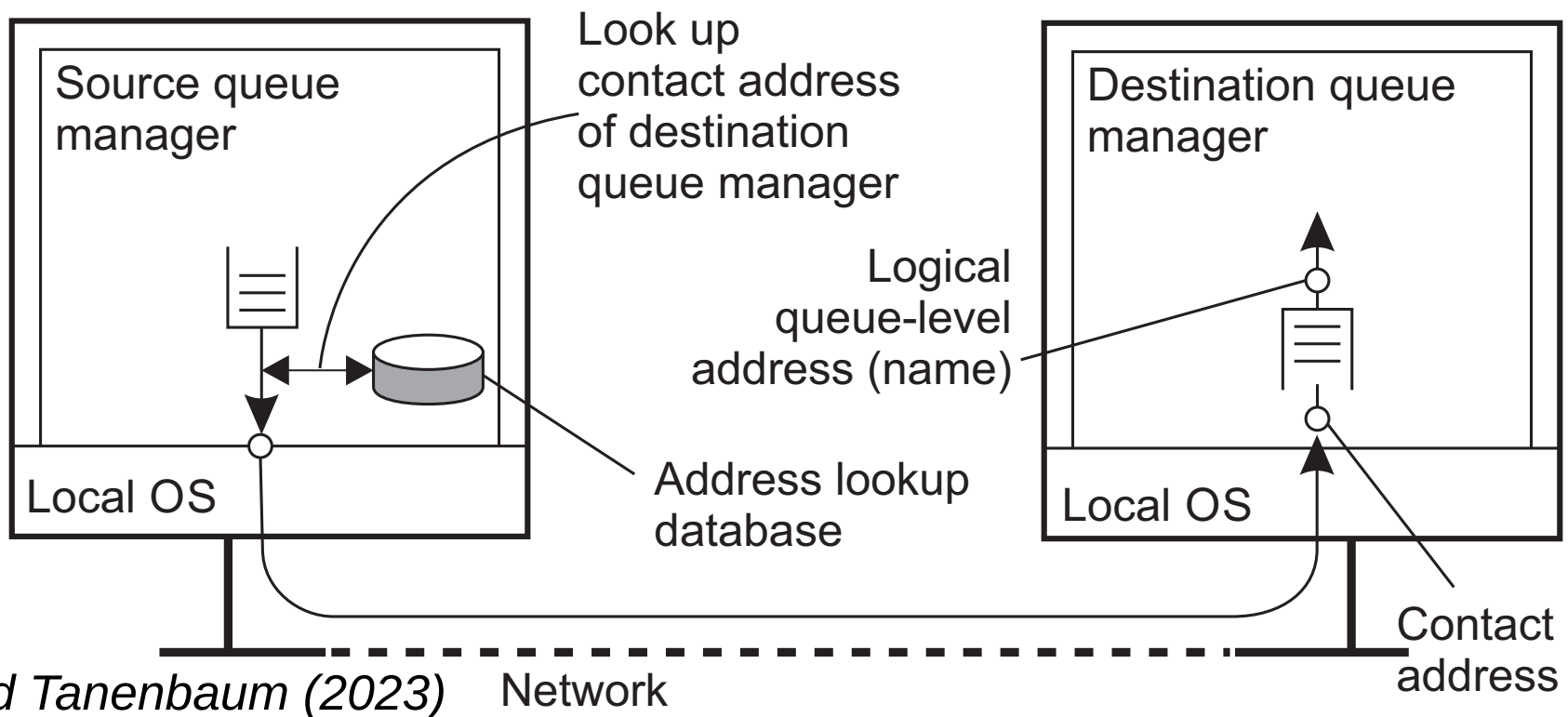
- **PUT**: A nonblocking operation to append a message to a specified queue
- **GET**: A blocking call that removes the longest-pending message from the queue
- **POLL**: A nonblocking variant of GET that checks for messages and returns immediately if the queue is empty
- **NOTIFY**: Installs a handler (callback) to be automatically triggered when a message arrives in the queue

MOM: General Architecture

The system is physically organized around **Queue Managers**, which are processes responsible for managing local queues and routing messages.

- **Local vs. Remote:** Applications normally interact only with their **local** queue manager
- **Name-to-Address Mapping:** Queue managers use lookup tables or directory services to map logical queue names to network addresses (e.g., host/port)
- **Routers:** In large-scale systems, specialized queue managers act as routers, forwarding messages across an application-level overlay network until they reach the destination manager

MOM: General Architecture



Steen and Tanenbaum (2023) Network

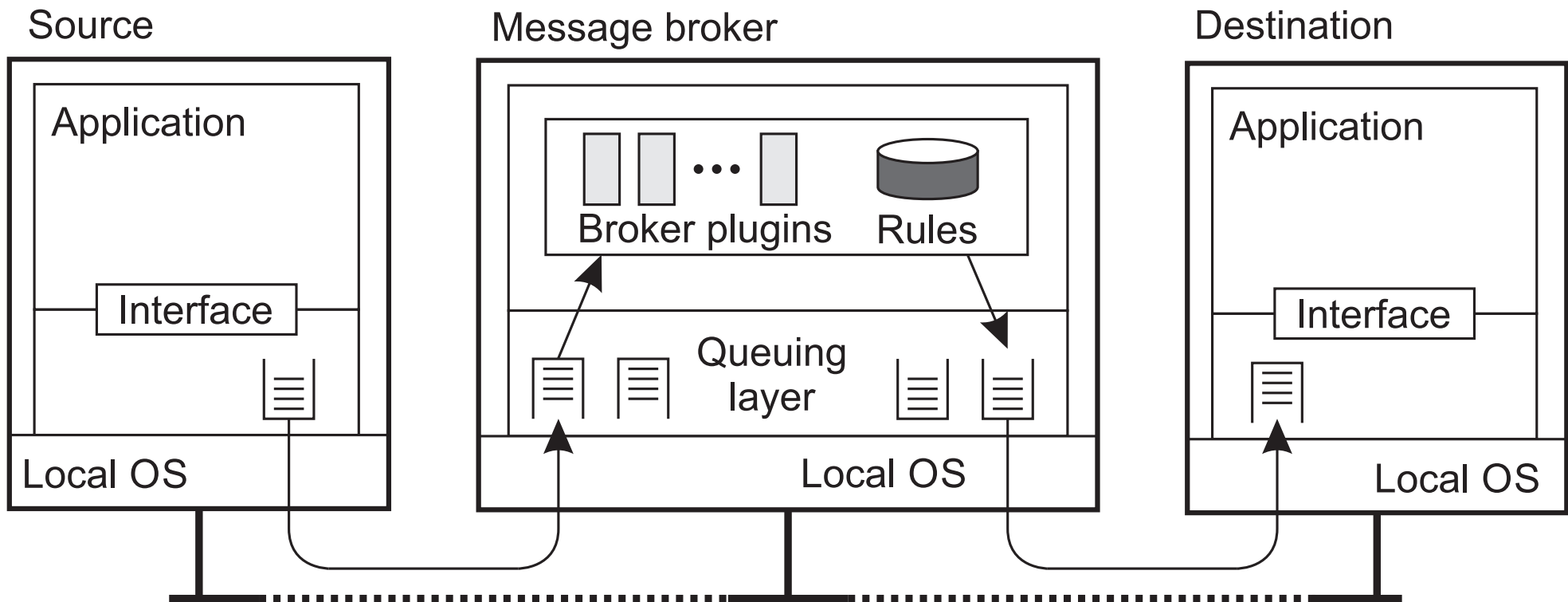
MOM: Message Brokers

Message brokers act as application-level gateways to solve the problem of heterogeneous message formats and semantics.

- **The Problem:** In a system with N applications, direct integration might require $N \times N$ protocol converters.
- **The Solution:** A broker acts as a central hub that converts incoming messages into the format and semantics expected by the receiver
- **Functionality:**
 - **Reformatting:** Changing delimiters, data types, or message layouts
 - **Content-Based Routing:** Directing messages to different queues based on their data

MOM: Message Brokers

Steen and Tanenbaum (2023)

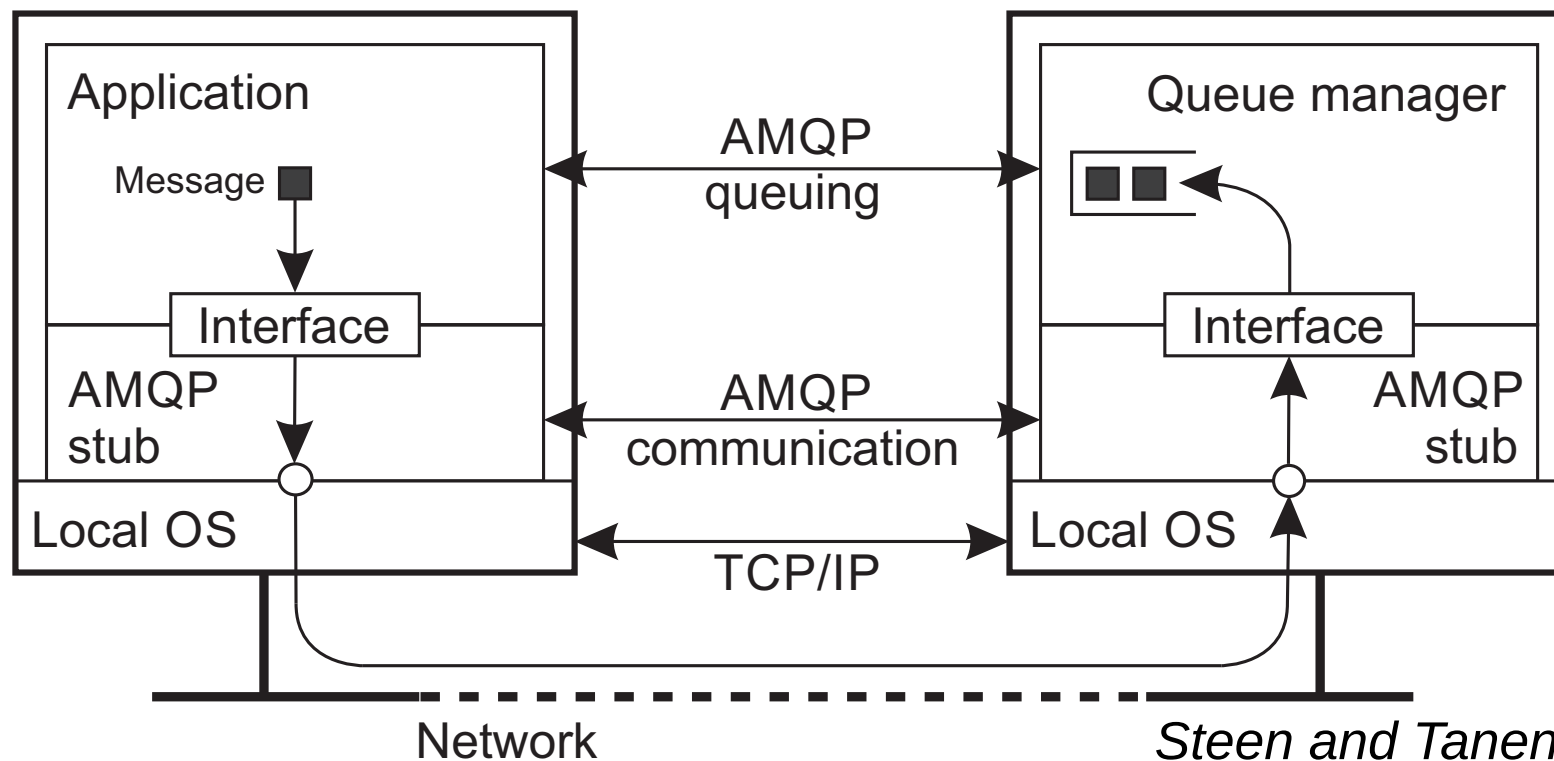


Advanced Message Queuing Protocol (AMQP)

The **Advanced Message Queuing Protocol (AMQP)** is an open standard protocol for message-oriented middleware designed to ensure interoperability between different platforms and languages

- **Goal:** Provide a wire-level protocol that allows diverse systems to exchange messages reliably, regardless of the underlying vendor or implementation
- **Strict Layering:** AMQP differentiates between the physical connection, the logical session, and the message transfer mechanism
- **Versatility:** It supports multiple communication patterns including point-to-point, request-reply, and publish-subscribe

Advanced Message Queuing Protocol (AMQP)



Steen and Tanenbaum (2023)

AMQP: Connections, Sessions, and Channels

AMQP organizes communication into a hierarchy to optimize network resources through multiplexing.

- **Connection:** A physical, full-duplex transport-layer link (typically TCP/IP) between two AMQP containers (applications or brokers)
- **Session:** A logical, bidirectional context established over a connection. It manages the sequential delivery and flow control of messages
- **Channel:** A mechanism for multiplexing a single connection into multiple independent threads of communication. This allows many logical conversations to share one physical TCP link, reducing overhead

AMQP: Nodes and Links

In AMQP 1.0, the core entities for message transfer are nodes and links.

- **Nodes:** Named entities within a container that act as sources or targets for messages. Examples include producers (sources) and consumers/queues (targets).
- **Links:** A unidirectional path between a source node and a target node
 - A **Source** is the node from which messages are sent
 - A **Target** is the node that receives the messages
- **Flow Control:** Links manage the transfer of “settled” or “unsettled” messages, ensuring the receiver is not overwhelmed

AMQP: Implementations and Ecosystem

While AMQP is a standardized protocol, its practical application is divided between two major versions: the feature-rich **AMQP 0-9-1** (popularized by RabbitMQ) and the ISO-standardized **AMQP 1.0** which focuses on wire-level interoperability across different vendors.

- **AMQP 0-9-1**: Prescribes a specific broker model involving Exchanges, Queues, and Bindings. It is highly optimized for complex routing logic within the broker itself
- **AMQP 1.0**: An “agnostic” wire protocol. It does not dictate how the broker is internally structured, allowing it to bridge disparate systems like JMS (Java Message Service) and cloud queues more effectively

AMQP: Implementations and Ecosystem

Broker	Protocol Support	Distinguishing Features
RabbitMQ	0-9-1, 1.0, MQTT	The most popular implementation of 0-9-1; focuses on flexible routing via exchanges.
Apache Qpid	1.0 (Native)	The Apache reference implementation for AMQP 1.0; designed for high-performance and cross-platform compatibility.
Apache ActiveMQ	0-9-1, 1.0, STOMP	A high-performance, non-blocking broker that supports multiple protocols and high availability.

Multicast Communication

Multicast Communication

Multicast communication is the process of sending data from a single source to a specific group of multiple receivers. While traditional networking handled this at the network layer, modern distributed systems often implement multicasting at the **application layer** (overlay networks) for better flexibility and manageability.

- **Targeting:** A subset of nodes (group) rather than all nodes (broadcast)
- **Infrastructure:** Utilizes overlay networks where nodes act as logical routers to disseminate information
- **Challenges:** Setting up efficient communication paths, handling group membership, and minimizing network overhead

Application-level Tree-based Multicasting

Nodes organize themselves into a logical tree structure where a unique overlay path exists between every pair of nodes.

- **Tree Construction (Chord Example):**

- A node initiates a session by generating a **multicast identifier** (mid).
- The node responsible for that identifier ($\text{succ}(mid)$) becomes the root of the tree.
- Nodes wishing to join send a $\text{lookup}(mid)$ request; as this request routes toward the root, intermediate nodes become “forwarders” and record the requester as a child.

- **Robustness vs. Efficiency:** Trees are efficient for dissemination but brittle; if a node fails, the subtree is partitioned until the network is reorganized.

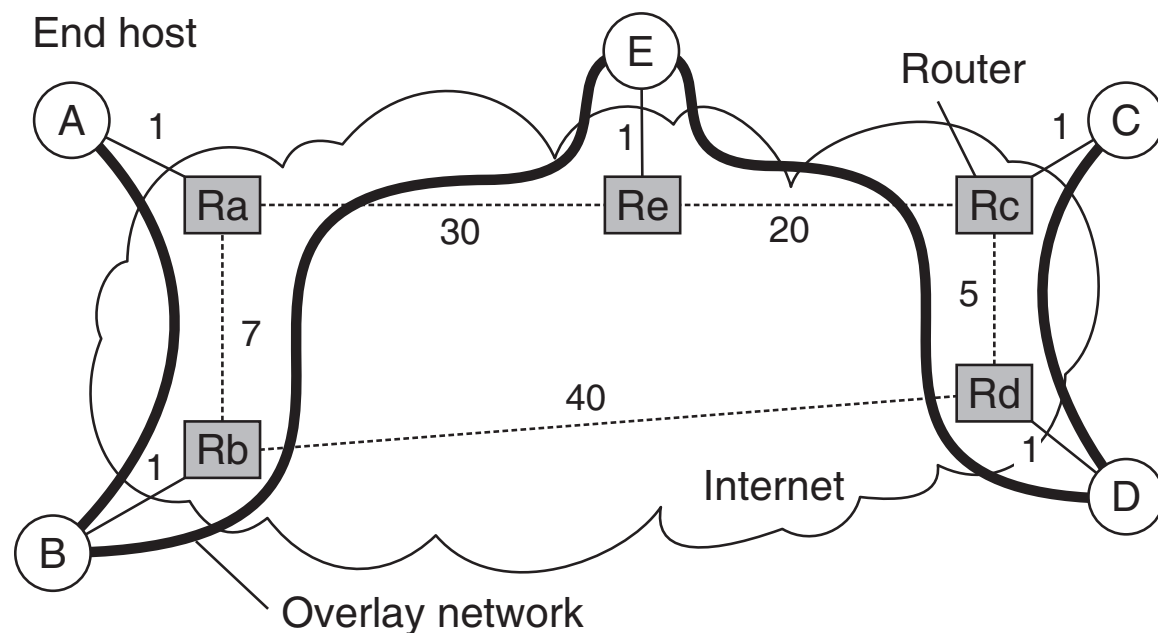
Performance Metrics for Multicast Trees

Because application-level trees are built on top of a physical network, their efficiency is measured by how well they map to the underlying infrastructure.

Metric	Definition & Significance
Link Stress	Counts how often a single physical link is traversed by the same message. High stress indicates inefficient mapping.
Stretch (RDP)	The ratio of the delay between two nodes in the overlay to the delay in the underlying physical network. Goal: Keep RDP close to 1.
Tree Cost	A global metric minimizing the total aggregated cost (e.g., total delay or bandwidth) of all links in the tree.

Performance Metrics for Multicast Trees: Link Stress

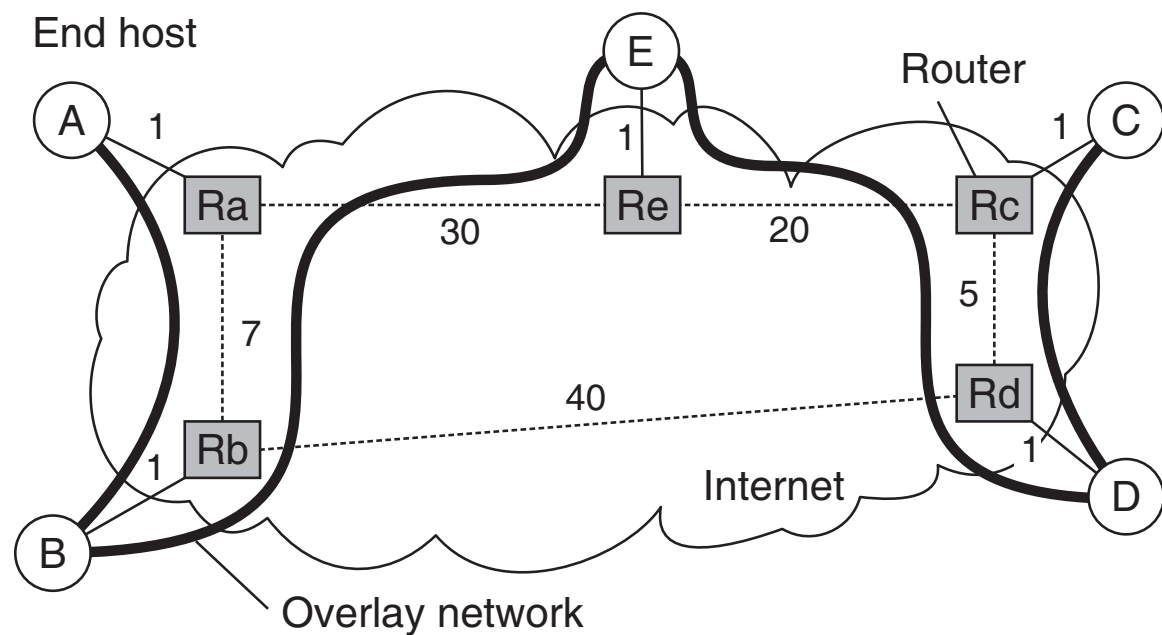
- A is the root of the multicast
- A multicast message to the other nodes will take the *logical* path:
 - $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C$
- The actual path is:
 - $A \rightarrow Ra \rightarrow Rb \rightarrow B \rightarrow Rb \rightarrow Ra \rightarrow Re \rightarrow E \rightarrow Rc \rightarrow Rd \rightarrow D \rightarrow Rd \rightarrow Rc \rightarrow C$
- Traversed twice:
 - $\langle B, Rb \rangle, \langle Ra, Rb \rangle, \langle E, Re \rangle, \langle Rc, Rd \rangle, \langle D, Rd \rangle$



Steen and Tanenbaum (2023)

Performance Metrics for Multicast Trees: Stretch

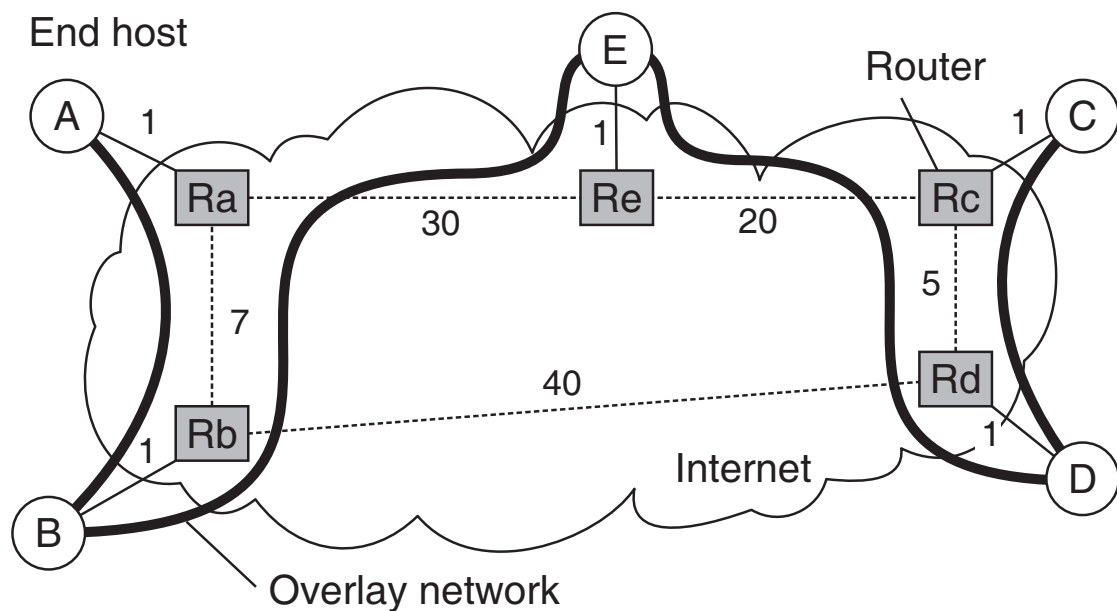
- Message from B to C has route:
 - $B \rightarrow Rb \rightarrow Ra \rightarrow Re \rightarrow E \rightarrow Rc \rightarrow Rd \rightarrow D \rightarrow Rd \rightarrow Rc \rightarrow C$
 - with total cost of 73 units
- The actual route is from B to C is:
 - $B \rightarrow Rb \rightarrow Rd \rightarrow Rc \rightarrow C$
 - with total cost of 47 units
- Hence, the stretch is:
 - $\frac{73}{47} = 1.553191489361702$



Steen and Tanenbaum (2023)

Performance Metrics for Multicast Trees: Tree Cost

- It is a measure used to evaluate a multicast tree, aimed at minimizing the combined costs of all active links
 - e.g., latency or bandwidth
- By minimizing the total cost, the system ensures that information dissemination across all nodes is as efficient as possible, similar to finding a **Minimum Spanning Tree (MST)**



Steen and Tanenbaum (2023)

Flooding-based Multicasting

In flooding, a node sends an incoming message to all its neighbors except the one it received it from.

- **Deterministic Flooding:**

- Every node forwards the message once
- To prevent infinite loops, nodes track unique message identifiers and discard duplicates
- In a fully connected graph of N nodes, this leads to $O(N^2)$ messages, which is inefficient

- **Probabilistic Flooding:**

- A node forwards the message to its neighbors with a probability
- This reduces network load but carries the risk that some nodes may never receive the message

Gossip-based Data Dissemination

Gossip-based data dissemination is often referred to as **epidemic protocols** because it mimics the way a biological virus or a rumor spreads through a population.

- **Objective:** Rapidly and reliably spread an update (a “virus”) to all nodes (the “population”) in a large-scale distributed system
- **Decentralization:** No central coordination exists; nodes make local, probabilistic decisions that lead to global system behavior

Gossip-based Data Dissemination: Node States

The behavior is modeled by classifying nodes into three distinct states based on their knowledge of a specific update:

- **Susceptible:** A node that has not yet received the update but is capable of being “infected”
- **Infected:** A node that has received the update and is actively attempting to spread it to others
- **Removed:** A node that has the update but has stopped spreading it
 - e.g., because it assumes everyone else already has it

Gossip-based Data Dissemination: Anti-Entropy Model

The “infection” spreads through periodic, pairwise interactions known as **anti-entropy**. A node selects a random node and synchronizes data using one of three strategies:

- **Push**: If P is infected, it sends the update to Q
 - If Q was susceptible, it becomes infected
- **Pull**: P contacts Q and asks for any updates it doesn't have
 - If Q is infected and P is susceptible, P becomes infected
- **Push-Pull**: A bidirectional exchange that ensures both nodes end up with the same set of updates
 - This is the most efficient method for rapid dissemination

Gossip-based Data Dissemination: Anti-Entropy Model

Properties of Epidemic Behavior

- **Exponential Spread:** The number of infected nodes grows exponentially in the initial stages, allowing an update to reach N nodes in $O(\log N)$ rounds
- **Robustness:** The system is highly resilient to node failures and network partitions
 - If a few “infected” nodes crash, many others still exist to continue the spread
- **Scalability:** Because nodes only interact with a small, constant number of random neighbors, the load on any single node remains independent of the total system size.

Gossip-based Data Dissemination: Rumor Spreading Model

Rumor spreading, a.k.a. **gossiping**, is a variation of epidemic protocols designed for the rapid dissemination of new updates across a distributed system. Unlike **anti-entropy**, rumor spreading is effective for injecting new information into a large number of nodes quickly.

The process is initiated when a node receives a new update (the “rumor”) for the first time.

- **Active Spreading:** The node becomes “infected” and starts periodically picking a random neighbor to push the rumor to
- **Interaction:** If the target node was “susceptible” (had not heard the rumor), it also becomes “infected” and begins spreading the rumor further
- **Feedback Loop:** If the target node was already “infected” (it had already heard the rumor), the sender receives feedback that the rumor is already known

Rumor Spreading Model: Becoming “Removed”

To prevent rumors from circulating indefinitely and wasting bandwidth, the protocol includes a termination condition where nodes move to a “removed” state.

- **Probabilistic Stopping:** If node P tries to push a rumor to node Q , and Q already knows it, P may decide to stop spreading the rumor with a probability of $\frac{1}{k}$.
- **State Transition:** Once a node stops spreading, it is “removed.”
 - It retains the data but no longer actively participates in the dissemination of that update
- **The “k” Factor:** The value of k determines the resilience of the spread
 - A higher k ensures the rumor reaches more nodes but increases the total number of messages sent

Rumor Spreading Model: Performance and Coverage

Rumor spreading is mathematically guaranteed to reach almost all nodes, though it is fundamentally probabilistic. The proportion of the population that **remains ignorant** is given by

$$s = e^{-(k+1)(1-s)}$$

- s : the proportion of the population that remains ignorant (the “missed” nodes)
- $1 - s$: the proportion of the population that eventually hears the rumor (the “coverage”)
- k : the “stubbornness” factor
 - If $k = 1$, a node stops spreading the first time it hits a neighbor who already knows the rumor
 - If $k = 2$, it might take two or more “redundant” contacts to make it stop

Rumor Spreading Model: Performance and Coverage

We can't isolate s easily with basic algebra. However, we can look at the results for common values of k :

Stopping Constant k	Missed Nodes s	Coverage $(1 - s)$
$k = 1$	$\approx 20\%$	$\approx 80\%$
$k = 2$	$\approx 6\%$	$\approx 94\%$
$k = 3$	$\approx 2\%$	$\approx 98\%$
$k = 4$	$\approx 0.7\%$	$\approx 99.3\%$

Rumor Spreading vs. Anti-Entropy

Feature	Rumor Spreading	Anti-Entropy
Primary Goal	Fast dissemination of new updates.	Ensuring long-term consistency.
Activity	Triggered by new data; eventually stops.	Runs periodically and continuously.
Efficiency	Very high; avoids redundant transfers.	Higher overhead due to constant checking.
Guarantees	Probabilistic; may miss some nodes.	Deterministic; eventually all nodes synchronize.

Summary

Summary: Foundations

Communication in distributed systems has evolved from low-level packet switching to high-level abstractions that hide the complexities of the underlying network. The goal is to provide a seamless interface for processes to interact across heterogeneous environments.

Modern distributed systems rely on the concept of layered protocols, often following the ISO OSI reference model.

- **Protocols:** Define the rules for data exchange
- **Middleware:** Operates primarily at the application layer to provide high-level communication services, abstracting away the transport and network layers
- **Synchronicity:** Communication is categorized as **persistent** (stored by middleware) or **transient** (discarded if not deliverable), and **synchronous** or **asynchronous**

Summary: Remote Procedure Call (RPC)

RPC remains the most widely used model for synchronous communication, providing **access transparency**.

- **Mechanism:** Uses client and server stubs to make a remote call look like a local one.
- **Marshaling:** Converts data into a machine-independent format to handle architectural differences.
- **Advantage:** Simplifies development by allowing programmers to work with standard procedure signatures.

Summary: Message-Oriented Communication

When synchronous RPC is too restrictive, message-oriented models offer greater flexibility through decoupling.

- **Transient (Sockets/MPI)**: Requires both parties to be active; used for high-performance and real-time data exchange
- **Persistent (Message Queuing)**: Decouples sender and receiver in time; messages are stored in queues until the receiver is ready
- **Brokers**: Act as application-level gateways to transform message formats between different systems

Summary: Multicast Communication

Multicasting allows efficient data dissemination to a group of receivers, often implemented via application-layer overlays.

- **Tree-based**: Highly efficient but vulnerable to node failures
- **Flooding**: Simple but can lead to excessive network overhead
- **Gossip (Epidemic)**: Extremely robust and scalable; uses probabilistic methods to ensure data eventually reaches all nodes

Summary: Comparison of Communication Models

Model	Primary Use Case	Key Characteristic
RPC	Request-Reply interactions	Access Transparency
Message Queuing	Loose coupling / Persistence	Time Decoupling
Sockets / MPI	High-performance transient data	Low overhead
Gossip / Epidemic	Large-scale dissemination	Fault Tolerance

Conclusion

Selecting the best communication mechanism depends on the specific requirements for:

- **Transparency:** How much of the network should be hidden?
- **Reliability:** Is message persistence required?
- **Scalability:** How many nodes must receive the data?
- **Performance:** What are the latency and throughput constraints?

Resources

Resources

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.
- <https://zeromq.org/>
- <https://www.rabbitmq.com/>
- <https://grpc.io/>
- <https://activemq.apache.org/>
- <https://bytebatego.com/>
- <https://qpid.apache.org/>